

预言机模块

Jintao

2026 年 3 月 9 日

预言机合约概述

实现概述

已实现具有延迟更新、多签投票阈值、安全控制的预言机合约

核心特性:

- 延迟价格更新 - 新价格需等待延迟期后生效，防止即时操纵
- 多数据源聚合 - 支持中位数或优先级模式聚合价格
- 多签投票机制 - 手动价格设置需多管理员投票同意
- 紧急停止 - 紧急情况下可停止预言机
- 价格变化阈值 - 大幅价格波动需额外投票确认

合约架构



Owner: 权限管理

Admin: 手动更新价格

Data Sources: 价格数据源

其他合约: 获取价格

图: 预言机合约架构

核心数据结构

```
1 // 价格存储
2 uint256 public currentPrice;      // 当前生效价格
3 uint256 public pendingPrice;      // 待生效价格
4 uint256 public pendingPriceTimestamp; // 待生效价格时间戳
5 uint256 public delayPeriod = 1 hours; // 延迟期
6
7 // 投票机制
8 address[] public admins;
9 uint256 public requiredVotes;      // 所需投票数
10 mapping(address => bool) public hasVotedPrice;
11
12 // 安全配置
13 bool public emergencyStop;         // 紧急停止标志
14 uint256 public priceChangeThreshold = 500; // 5%变化阈值
15 PriceAggregationMode public currentMode; // 聚合模式
16
```

价格更新流程

- 1 触发条件 - `updatePrice()` 仅在距离上次更新超过 `delayPeriod` 后才能调用
- 2 数据聚合 - 从多个 Data Source 获取价格并聚合
- 3 阈值检查 - 检查新聚合价格相比当前 `pendingPrice` 的变化是否超过阈值
- 4 更新 **pendingPrice**
 - 变化 $\leq$ 阈值: 直接更新 `pendingPrice`
 - 变化 $>$ 阈值: 发起多签投票, 投票通过后更新 `pendingPrice`
- 5 更新 **currentPrice** - `currentPrice` 更新为上一个 `pendingPrice`

关键机制

- `updatePrice` 需间隔 `delayPeriod` 才能再次调用
- `currentPrice` 始终是上一个已生效的 `pendingPrice`
- 价格变化超过阈值需多签投票确认

触发时机设计

触发时机

`updatePrice()` 应在 **`delayPeriod`** ~ **`2×delayPeriod`** 范围内随机触发，而非精确间隔触发

随机触发原因：

- ① 网络延迟 - 精确间隔调用可能因网络拥堵导致交易失败
- ② 防止预测攻击 - 可预测的更新时间为攻击者提供操纵窗口
- ③ 减少 **MEV** 机会 - 降低套利者通过预测更新时间获利的可能性

延迟上限

必须在 **`2×delayPeriod`** 内触发，否则当前价格 (`currentPrice`) 失效

价格聚合模式

中位数模式 (MEDIAN)

- 收集所有有效数据源价格
- 计算中位数作为最终价格
- 抗单点故障

优先级模式 (PRIORITY)

- 按顺序尝试数据源
- 返回第一个有效价格
- 适合信任特定数据源

```
1 enum PriceAggregationMode {  
2     MEDIAN,      // 中位数  
3     PRIORITY    // 优先级  
4 }  
5
```

投票机制

支持投票的操作:

- `proposeManualPrice()` - 手动设置价格
- `proposeEmergencyStop()` - 紧急停止
- `proposePriceAggregationMode()` - 更改聚合模式
- `proposePendingPrice()` - 手动设置待生效价格
- `voteForLargePriceChange()` - 大幅价格变化确认

```
1 function voteForManualPrice() external onlyAdmin {  
2     require(isPriceVotingActive, "No active voting");  
3     require(!hasVotedPrice[msg.sender], "Already voted");  
4  
5     hasVotedPrice[msg.sender] = true;  
6     priceVoteCount++;  
7  
8     if (priceVoteCount >= requiredVotes) {  
9         currentPrice = proposedPrice;  
10        isPriceVotingActive = false;  
11    }  
12 }  
13
```

角色定义

- **Owner** - 合约所有者，可配置所有参数
- **Admin** - 管理员，可发起投票和更新价格
- **Automation** - 自动化角色，可触发价格更新

Owner 专属权限:

- 添加/移除数据源
- 添加/移除管理员
- 设置投票所需票数
- 设置价格变化阈值
- 授予/撤销 Automation 角色

合约安全性设计

防止攻击:

- 延迟生效机制
- 价格变化阈值
- 多签投票确认
- 紧急停止功能

容错机制:

- 多数据源聚合
- 备用聚合模式
- 投票取消机制
- 价格有效性检查

价格有效性

价格在 $\text{pendingPriceTimestamp} + 2 * \text{delayPeriod}$ 内有效

价格数据获取方式

预言机合约的价格数据来源主要有三种方式

- ① 自建节点构建预言机 - 完全自控，但成本高
- ② 自定义 **API** 数据上链 - 灵活度高，需公开 API
- ③ 第三方预言机服务 - 即插即用，如 Chainlink

方案一：自建节点构建预言机

不适用于本项目

自建预言机节点需要大量资源和维护工作

实施步骤：

- ① 部署预言机合约
- ② 实现数据签名机制
- ③ 开发链上验证合约
- ④ 建立节点监控和告警系统

成本估算：

- 服务器成本: 3-5 台服务器
- 开发成本: 2-3 个月开发时间
- 运维成本: 专人维护，长期投入

结论

成本高昂，维护复杂，不适合本项目

方案二：自定义 API 数据上链

概述

使用自有 API，通过第三方中间件定时将数据推送上链

前置条件：

- API 需可公开访问（或允许预言机节点 IP 访问）
- 具有稳定的可用性和低延迟

优势：

- 数据源完全可控
- 成本相对较低
- 灵活性高

第一方/源头签名

API3

- 部署 Airnode, 连接 API 和区块链
- 主要成本是区块链 Gas 费

RedStone

- 按照协议格式签名数据，广播至缓存层
- 成本来自数据存储（约 \$0.01/次）

Pyth Network

- 运行发布者客户端，用私钥签名数据
- 成本主要是链上 Gas 费。在支持的链上（如 Solana、Aptos）成本极低（每次更新 <0.001 ）。在 EVM 链上通过 Wormhole 传递，成本稍高但可控。

第三方网络执行

第三方网络执行

发起请求，通过各自的去中心化网络来获取和提交数据

Chainlink

- 使用 Functions 0.25–2+/次；
- Any API: 配置 Job，让节点定期获取你的 API 数据并推送到链上，每月数百至上万。

Witnet:

- 成本 \$0.05 - \$0.5/次

Tellor:

- 其 PoW 竞争模型导致单次更新成本高（5–20+）且速度慢，不适合有规律的低频更新。

Band Protocol:

- 在 Cosmos 生态内是优秀选择。如果在 EVM 或其他链上，其优势不明显

方案三：第三方预言机服务

概述

直接接入成熟的预言机网络获取价格数据

主流选择：

Chainlink

- 市场主导地位
- 成熟的喂价服务

Pyth Network

API3

- 只有 XAUT 价格数据，没有现货黄金价格

RedStone

数据源对比

特性	自建节点	自定义 API	第三方服务
成本	高	中	极低
维护难度	高	中	低
数据控制	完全	完全	依赖第三方
部署速度	慢	中	快
可用性保障	自行负责	自行负责	网络保障
安全性	自行负责	自行负责	依赖第三方

表: 数据源方案对比

建议

对于本项目，推荐使用第三方预言机服务（方案三），或结合自定义 API（方案二）作为补充数据源